towards
data science
The world's leading publication for data science, AI, and ML professionals.

DATA SCIENCE

Alternative Visualizations for Data-Driven Storytelling

Why to choose them, and how to create them

Michal Szudejko
Nov 2, 2023 • 19 min read

Source: image generated by the author in ChatGPT and DALL-e 3.

Did you know that all Formula 1 cars must incorporate a large wooden plank as one of their components?

> A wooden plank?
>
> Yes, indeed!

Formula 1 cars utilize a wooden plank, the **"skid block,"** positioned underneath the car. This ensures the car's height remains within the legally mandated limits during races. Moreover, the plank serves as a critical safety feature, sacrificing scraping against the track to shield the car's essential components from potential damage when contacting curbs. Importantly, its impact on performance is minimal. The skid block is crafted from a composite material of wood and fibers and is designed to wear at a steady pace.[1]

Why do I mention this? This anecdote highlights that even a technologically advanced field such as Formula 1 can effectively utilize a (*theoretically*) simple component . This parallel extends to the realm of data visualization. As I have argued on numerous occasions, **most data-driven insights can be adeptly communicated using three basic chart types: bar chart, line chart, and pie chart.**

But what if these three types fall short of capturing the essence of the phenomenon we're attempting to explain? Or what if we employ multiple simple charts instead of a single, but slightly more complex, one? Or, returning to our F1 example, we simply decided to replace wood with carbon fiber and draw something more fancy? **In this post, I will explore the alternatives available to you should such scenario occur.**

So, what can you expect from this post?

The goal is to provide an overview of alternative data visualization forms. **So, I'm going to talk about chart options beyond the traditional triumvirate of bar, line, and pie charts, which I consider the default choice for most cases anyway.**

That said, I fully recognize the wealth of other visualization options that are available. I usually consider using them when the conventional trio doesn't fit the bill. Here, I'll introduce you to a selection of visualization forms that I find particularly valuable. I will also guide you through some straightforward methods for creating them using various, practical, and cost-free tools. It's important to note that, for this article, I'm assuming that you'll be feeding your visualizations with data stored locally on your computer unless there are specific advantages to an alternative approach, which I will highlight. To be fair, I took writing this post as an opportunity to learn these tools and I must say, that it was a great fun to learn this stuff (apart from the practical value of that fact). **Hopefully, it will be for you as well!**

The decalogue of visualization

Before we explore the vast and diverse realm of data visualization, I'd like to introduce you to a novel idea I've developed: the **"Decalogue of Visualization."** This is a straightforward compilation of guidelines that can assist you in crafting a data-driven visual, regardless of whether you opt for a fundamental visualization format like a bar chart or decide to venture into the more intricate alternatives we will discuss subsequently.

1. **Start with the purpose.** Always start working on the visualization with a clear purpose in mind.
2. **Make eye tracking easier.** Direct the attention of the recipients towards the key message that stems from the purpose.
3. **Reduce cognitive load.** Remove all unnecessary elements that do not work for the purpose or make it difficult to track crucial information in the visualization.
4. **Choose precise visualization attributes**, such as shape, direction, or size, rather than hue or volume.
5. **Declutter.** Remove all technical elements of the chart unless they help points 1 to 4 of this decalogue.
6. **Maximize data-ink ratio.**
7. **Use color strategically.** Reduce the number of colors, and clearly differentiate between important and unimportant information in the chart.
8. **Design with integrity and consistency.** Consistently apply the style used for visualization across your presentation or report.
9. **Ensure inclusiveness.** E.g., take necessary care for people with sight impairments.
10. **Do not manipulate with visualizations.** Avoid manipulative techniques.

If you are interested in finding out more about these rules, please be welcome to visit my blog. I discuss these points in-depth and share some visual examples of what I mean by specific points. The article is in English & Polish.[2]

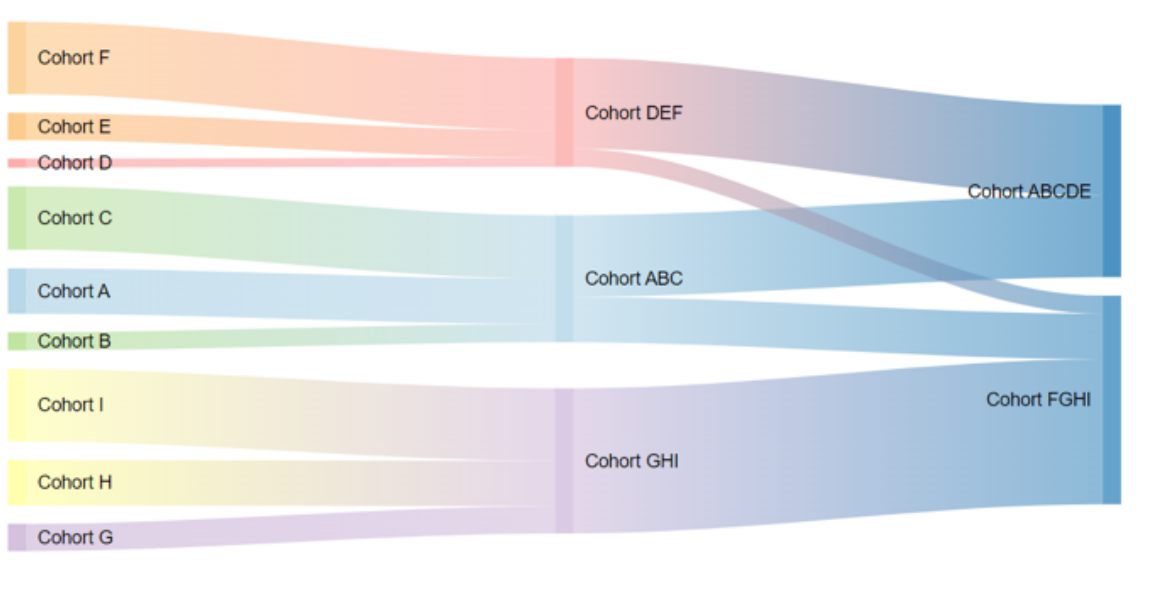
Alternative visualization forms

Alright, let's come back to our primary discussion. I've handpicked a few visualization types that deviate from the conventional bars, lines, and pies. I will delve into them in greater detail. Each of these visuals has been selected because it fulfills specific needs, which I will outline. Consider integrating these visualizations into your work whenever the basic chart types fall short. You may also select them if you simply want to break the monotony of the three foundational chart types. This may help to captivate your audience's attention freshly and engagingly.

A word of caution!

I've encountered countless data visuals labeled as alternative, creative, or innovative by their creators. And to be fair, they had every right to make that claim. However, from my perspective, most of these visuals lacked practical application. They tended to be overly complex, requiring substantial effort to extract meaningful insights, or were too decorative. Consequently, I might be a bit conservative while selecting alternative visuals. **Rest assured, each visualization I've chosen has been rigorously "field-tested" by myself.** Therefore, I can confidently attest for their practicality and applicability, particularly within a corporate environment.

Sankey diagram



Sankey diagram. Source: diagram generated in Google Charts by the author.

A **Sankey diagram**[3] is a specialized type of flow diagram characterized by connectors whose thickness is proportional to the volume of flow they represent. These diagrams are commonly utilized for both scientific and business applications.

A Sankey diagram is comprised of two primary components:

- **Nodes** which symbolize various states or stages within the flow process.
- **Connectors**, which depict the flow from one node to another.

The width of each connector is directly proportional to the total flow volume between the corresponding nodes.

Sankey diagrams can be employed for a range of purposes, including:

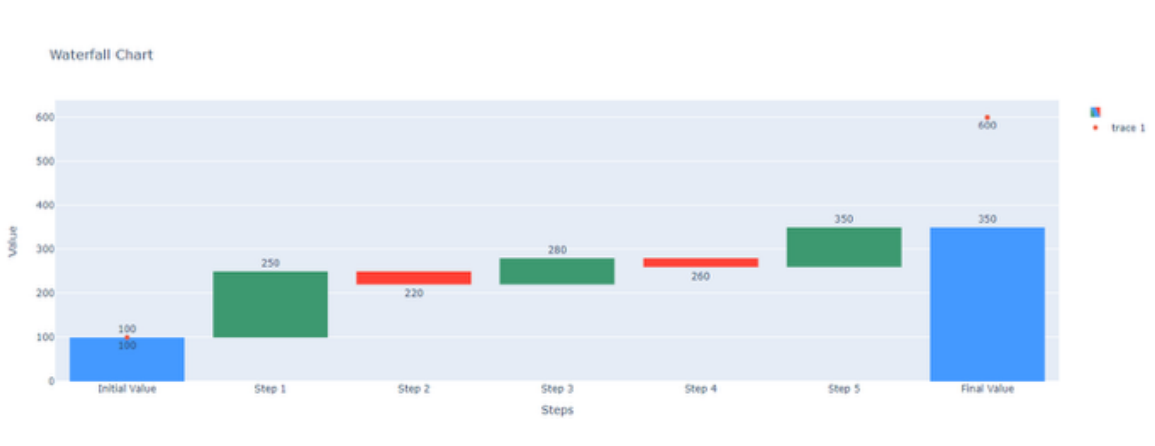
- **Tracking customer migrations**, such as transition from one CRM system to another.
- **Analyzing cash flows** within a business or the broader economy.
- **Examining information** flow within a computer network.
- **Investigating material usage** throughout a production process.

I utilized **Google Charts**, a comprehensive charting service developed by the tech giant Google / Alphabet, to draw the above diagram. Google Charts is a JavaScript library that enables users to create charts based on their data inputs. Once the necessary information is inputted and formatting preferences are specified, Google Charts generates the chart and provides the HTML/JavaScript code required to embed the visualization into a website.

Below is the code I generated for the chart presented above. It is a copy-paste of few code elements from the Google Charts site plus my own data. This code can be saved as a plain HTML, embedded to the site as I did above. You can also execute it in tools such as JSFiddle.

```
<head>
    <script type="text/javascript" src="https://www.gstatic.com/chart
    <script type="text/javascript">
      google.charts.load('current', {'packages': ['sankey']});
      google.charts.setOnLoadCallback(drawChart);

      function drawChart() {
        var data = new google.visualization.DataTable();
        data.addColumn('string', 'From');
        data.addColumn('string', 'To');
        data.addColumn('number', 'Weight');
        data.addRows([
          ['Cohort A','Cohort ABC',5],
          ['Cohort B','Cohort ABC',2],
          ['Cohort C','Cohort ABC',7],
          ['Cohort D','Cohort DEF',1],
          ['Cohort E','Cohort DEF',3],
          ['Cohort F','Cohort DEF',8],
          ['Cohort G','Cohort GHI',3],
          ['Cohort H','Cohort GHI',5],
          ['Cohort I','Cohort GHI',8],
          ['Cohort ABC','Cohort ABCDE',9],
          ['Cohort ABC','Cohort FGHI',5],
          ['Cohort DEF','Cohort ABCDE',10],
          ['Cohort DEF','Cohort FGHI',2],
          ['Cohort GHI','Cohort FGHI',16],
        ]);
    var colors = ['#a6cee3', '#b2df8a', '#fb9a99', '#fdbf6f',
                  '#cab2d6', '#ffff99', '#1f78b4', '#33a02c'];
        // Sets chart options.
        var options = {
          width: 600, height: 290,
      sankey: {
        node: {
          colors: colors
        },
        link: {
          colorMode: 'gradient',
          colors: colors
        }
      }
        };

        // Instantiates and draws our chart, passing in some options.
        var chart = new google.visualization.Sankey(document.getElem
        chart.draw(data, options);
      }
</script>
</head>
  <body>
    <div id="sankey_basic" style="width: 600px; height: 300px;"></div
  </body>
</html>
```

Waterfall chart



Waterfall chart generated in Google Collab. Source: image by the author.

Waterfall charts are "very popular" among finance specialists such as budget controllers or financial analysts. Personally, I hate it because I have seen it so many times. Yet, I use waterfalls because I see a lot of objective value in them.

A waterfall chart is a graphical representation, particularly in budget Presentations or financial reports. This chart is instrumental in illustrating the breakdown of the variation between two points or scenarios (such as execution and budget), highlighting its constituents.

The waterfall chart is designed to showcase how an initial value evolves through a sequence of positive and negative alterations, culminating in a final value. It is composed of columns that can be organized either vertically or horizontally. These columns represent the individual factors contributing to the initial value shift. Typically, positive changes are denoted by green columns, while red columns represent negative changes. The first column of the chart displays the starting value, the final column indicates the ending value, and the intervening columns delineate the individual changes that facilitate the transition from the initial to the absolute value.

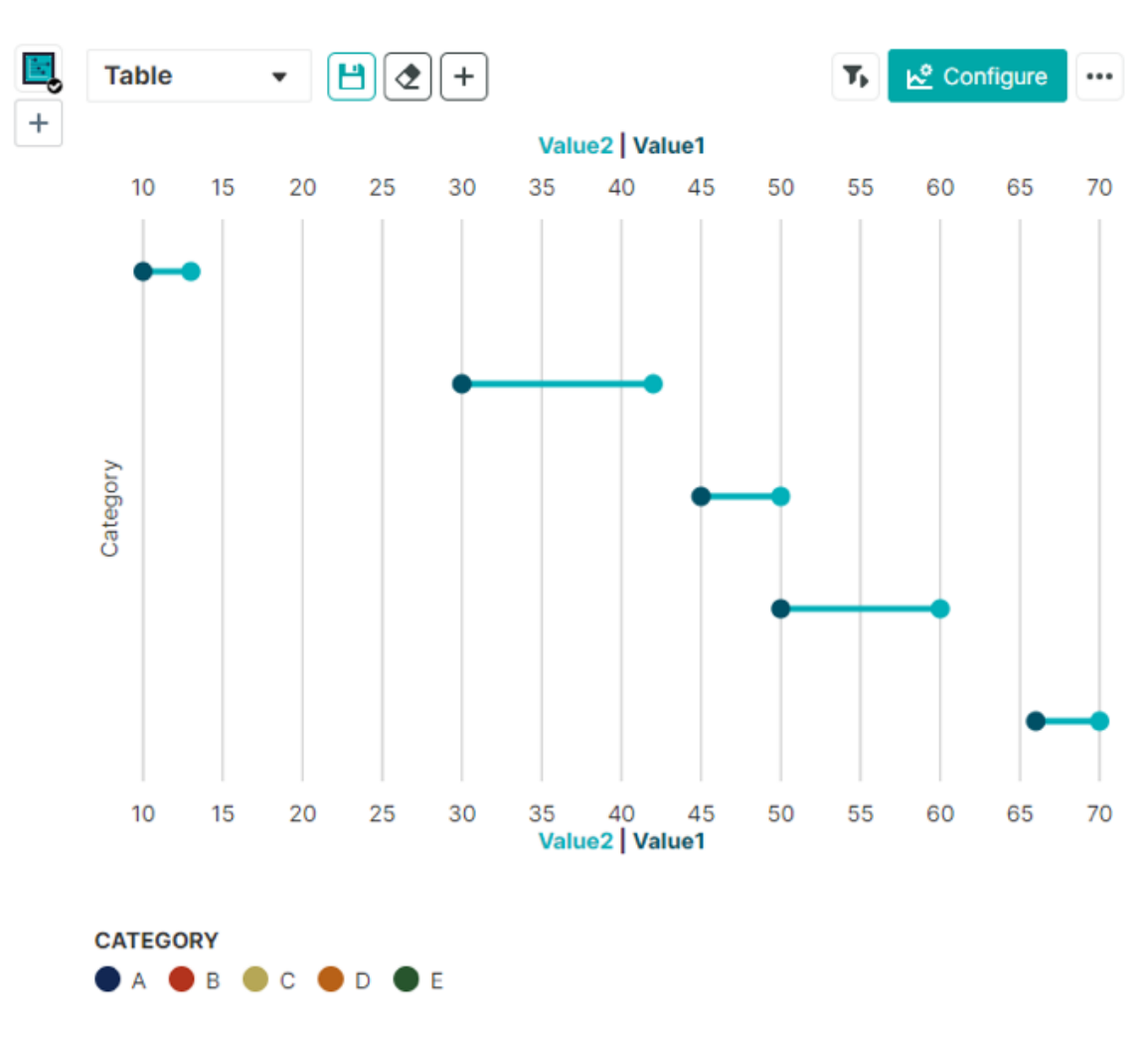
Below is the Python code for the waterfall chart, utilizing the Plotly library. I used ChatGPT (GPT 3.5) to craft this code.

```
import plotly.graph_objects as go

# Input data - sample values for different steps
steps = ['Initial Value', 'Step 1', 'Step 2', 'Step 3', 'Step 4', 'St
values = [100, 150, -30, 60, -20, 90, 250]  # Sample changes in value

# Calculate the final value based on the input data
final_value = sum(values)

# Create the waterfall chart
fig = go.Figure(go.Waterfall(
    name="",
    orientation="v",
    measure=["absolute", "relative", "relative", "relative", "relativ
    x=steps,
    textposition="outside",
    texttemplate="%{y}",
    y=values,
    connector={"mode": "spanning", "line": {"width": 0}},
))

# Add initial and final values to the chart
fig.add_trace(go.Scatter(x=['Initial Value', 'Final Value'], y=[100, 
                         text=[str(100), str(final_value)], textposit

# Configure the chart
fig.update_layout(
    title="Waterfall Chart",
    xaxis_title="Steps",
    yaxis_title="Value",
)

# Save the chart to an HTML file
fig.write_html("waterfall_chart.html")

# Display the chart
fig.show()
```

The code was executed in Google Collab. You can see the final product at the beginning of this paragraph.

If you don't want to use Google Collab and keep the data locally, I recommend using **Anaconda Distribution**. Specifically, I pasted the following code to **Spyder IDE**, installed within Anaconda. It might be required to install the Plotly library, which you can do in **Anaconda Powershell Prompt** (use the following command line: *pip install plotly* or *pip install -U plotly* to get the latest version).

Dumbbell plot



Example of a dumbbell plot. Source: print screen by the author.

A **dumbbell plot**[4] is a type of chart designed to illustrate changes in values between two distinct points in time or across two categories. This Visualization technique is precious when one aims to compare the differences between two data points across varying categories or entities.

The structure of a dumbbell plot includes two distinct points for each category (or unit) connected by a line. These points show values at two separate moments in time or within two disparate categories. The connecting line represents the change in value between these two points, facilitating a straightforward comparison of which categories experienced the most significant changes over time or which units varied the most between the two categories.

Here are a few examples that illustrate the potential applications of a dumbbell plot:

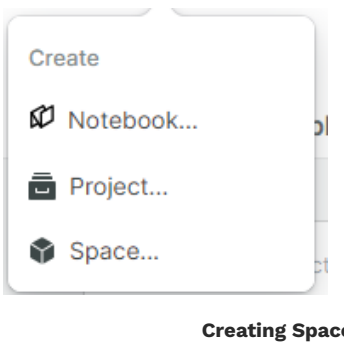
1. **Product price changes:** This can be employed to compare product prices across two different years, with one end of the dumbbell representing the price in the first year and the other in the second year.
2. **Comparison of test results in medicine:** Dumbbell plots can be used to compare patients' diagnostic test results before and after treatment.
3. **Changes in production:** For companies looking to assess their production performance across two different quarters, a dumbbell plot can succinctly compare the number of units produced in each respective quarter.
4. **Changes in employee ratings:** Human Resource departments can leverage dumbbell plots to compare employee ratings across two assessment periods.
5. **Sales comparison:** Marketing departments can employ dumbbell plots to assess a product's sales performance before and after the launch of an advertising campaign.

I designed the above dumbbell plot using **Noteable** and **Data Prism**. Data Prism is built into Noteable.[5]

Noteable is an analytics platform and collaborative data workspace. It is accessible as a website and an integrated add-on to ChatGPT.

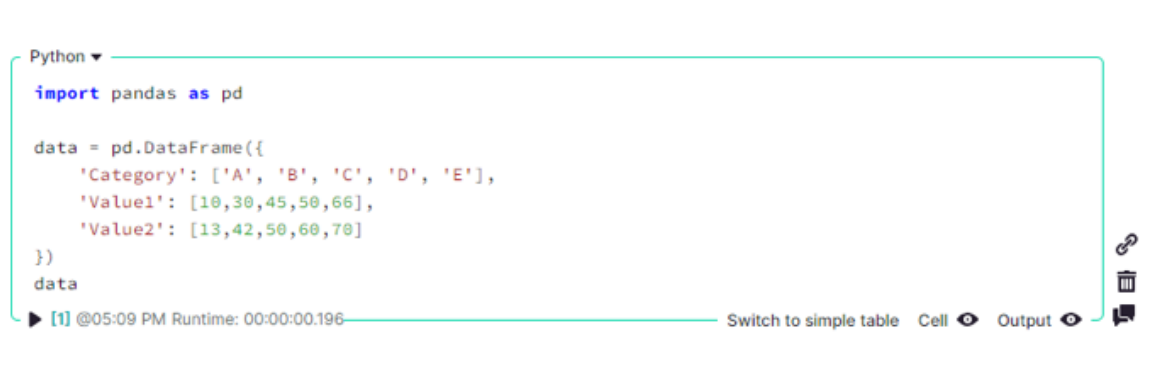
One potential drawback of creating charts using Noteable is the limitation that, to the best of my knowledge, charts cannot be replicated locally. Nonetheless, the platform is well worth exploring, mainly because it incorporates the innovative Data Prism feature. This tool enables users to generate many visualizations for their data at the simple click of a mouse. The ease and speed with which one can experiment with different visualizations enhances the likelihood of uncovering previously unnoticed trends, dependencies, or outliers in the data.

As a first step, we need to create a space & notebook in Noteable.



Creating Space & Notebook in Noteable. Source: print screen by the author.

Then, in the newly created notebook, we can input our data. Here I created a dataframe, but you could upload a *.csv (comma-delimited) file as well (first you need upload such a file to the project and then add to notebook).



Python ▾

```
import pandas as pd

data = pd.DataFrame({
    'Category': ['A', 'B', 'C', 'D', 'E'],
    'Value1': [10,30,45,50,66],
    'Value2': [13,42,50,60,70]
})
data
```

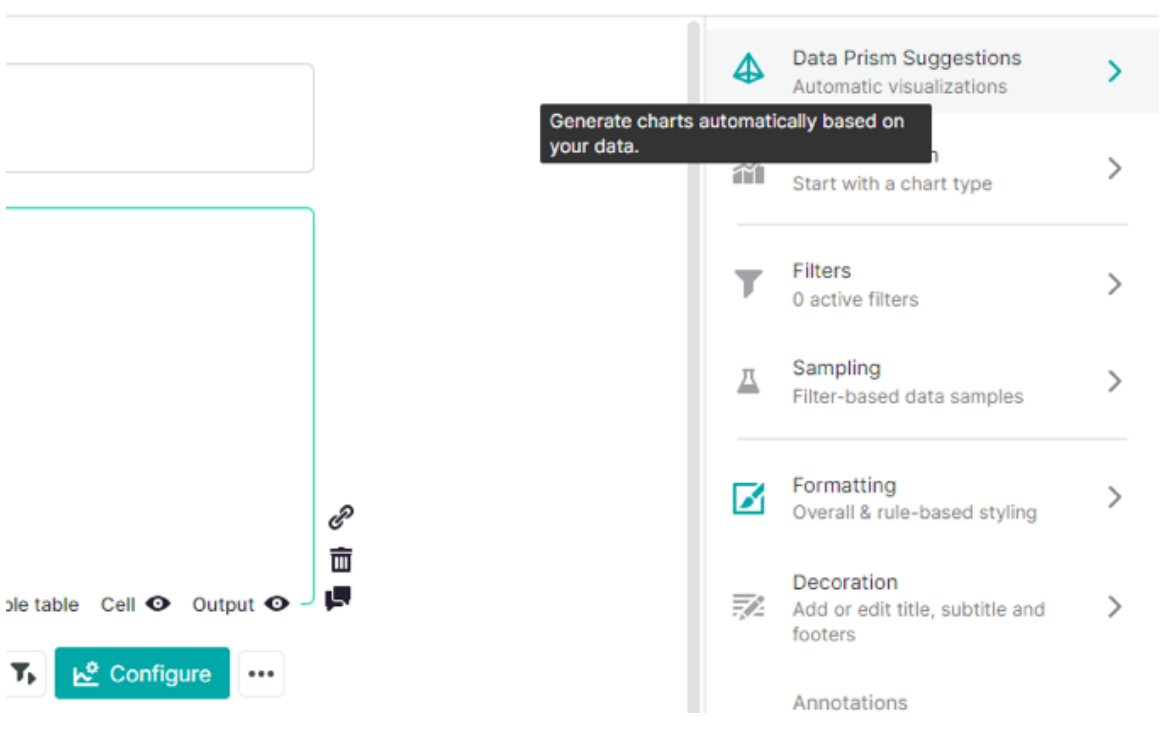
▶ [1] @05:09 PM Runtime: 00:00:00.196

Data input to Noteable. Source: print screen by the author.

And here is the code shown above.

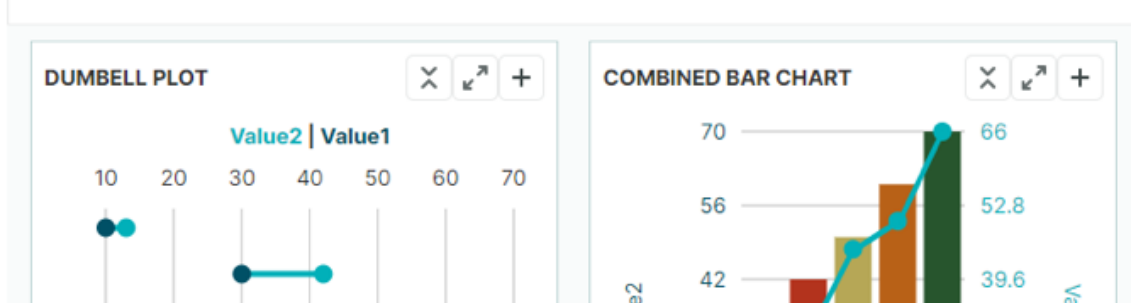
```
import pandas as pd
data = pd.DataFrame({
    'Category': ['A', 'B', 'C', 'D', 'E'],
    'Value1': [10,30,45,50,66],
    'Value2': [13,42,50,60,70]
})
data
```

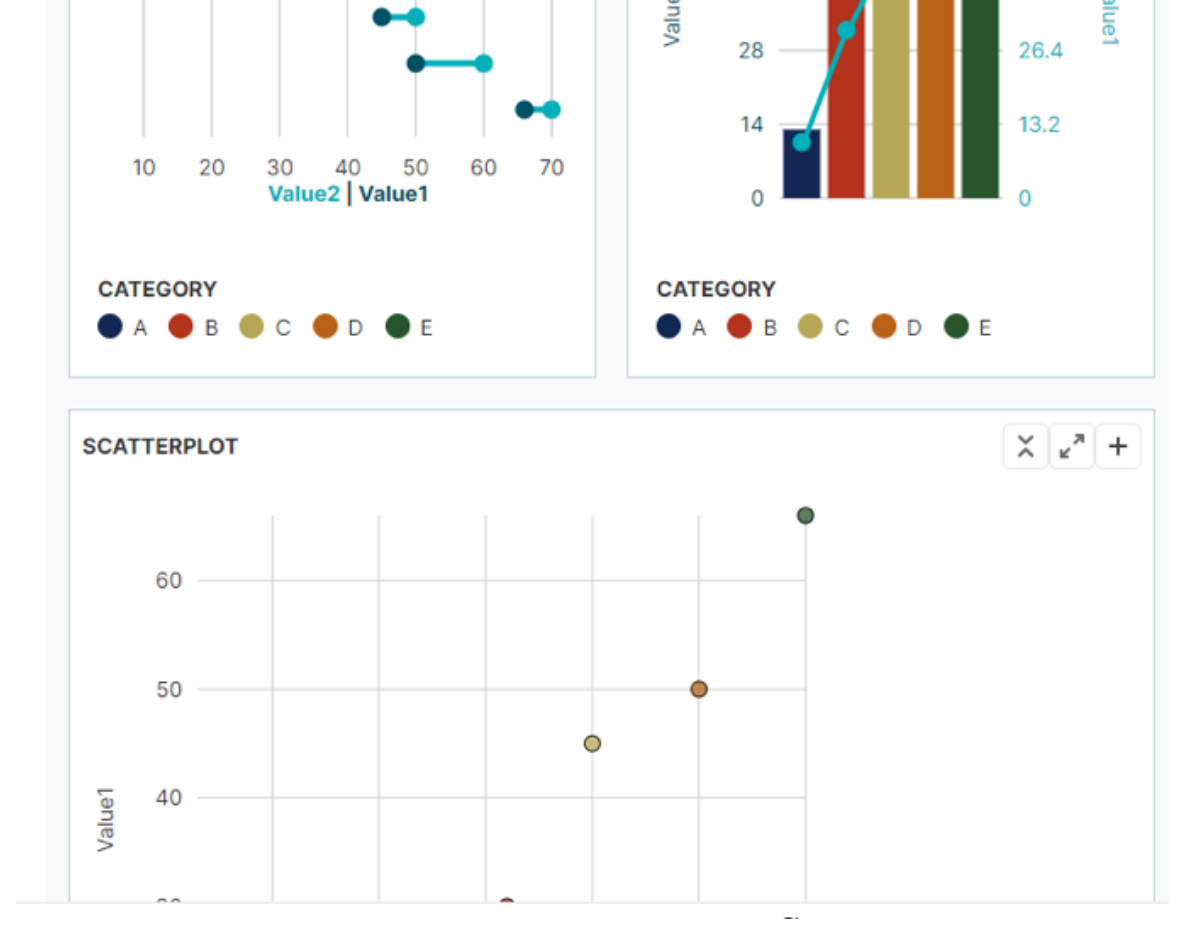
Next, we click on "Configure" and then "Data Prism Suggestions".



Launching Data Prism suggestions. Source: print screen by the author.

Then we get a number of options from which we can choose and further work on.

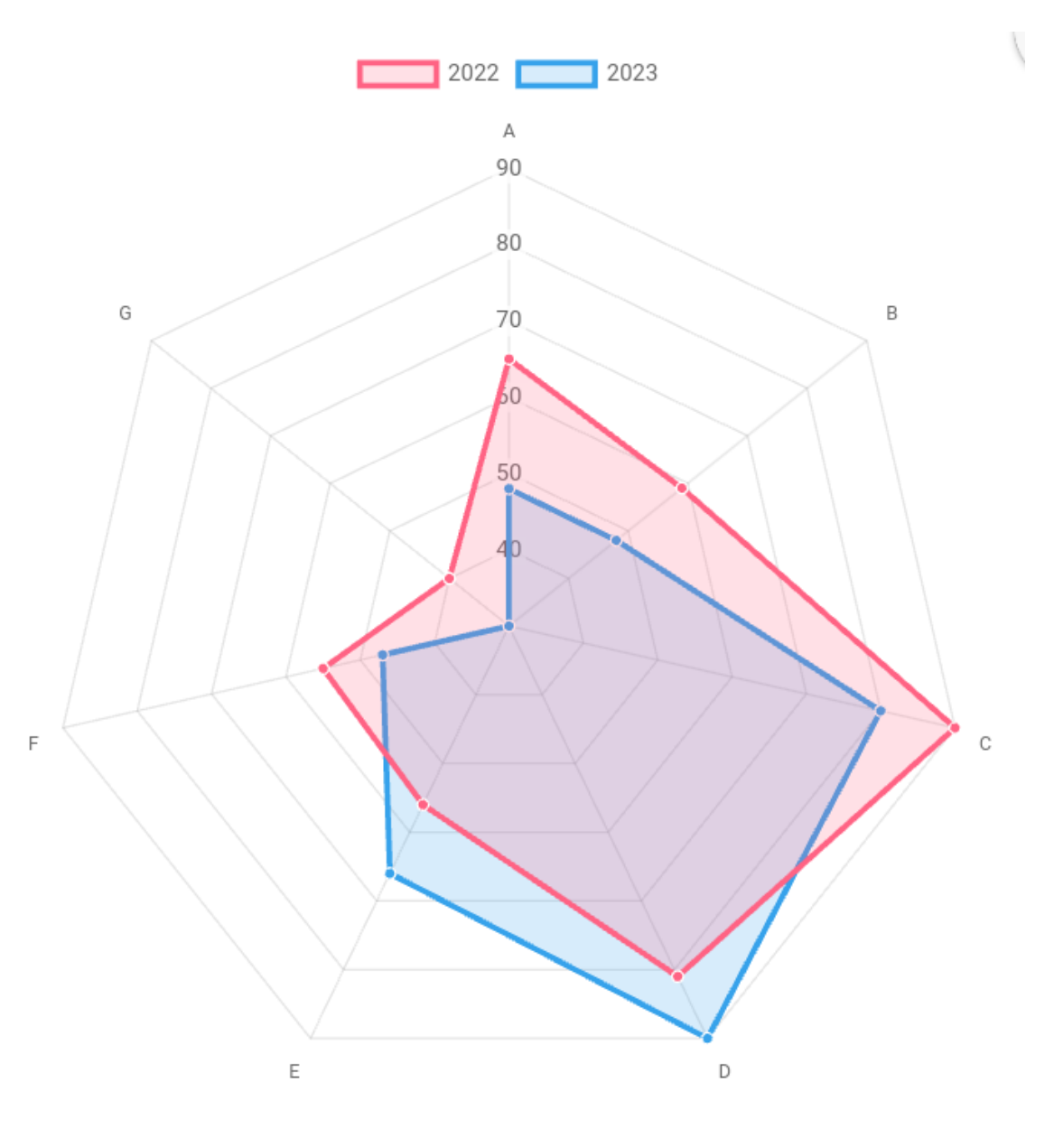




Charts suggested by Data Prism. Source: image by the author.

Lastly, we can export our chart to *.png (unfortunately, it does not work well unless that's a problem on my computer). We can also publish it and then embed on our website.

Radar chart



Radar chart crafted in chart.js. Source: print screen by the author.

A **radar chart** is a visual used to display multivariate data in a two-dimensional graph format, where three or more quantitative variables are represented on axes emanating from the same central point. These axes are evenly distributed radially around a circle, and the values of each variable are plotted along these axes. The plotted points are then connected by a line to form a shape. The shape formed by the line and the positions of the points in relation to the axes reveals insights about the data.

Here are some practical applications for radar charts:

1. **Sports performance analysis:** Radar charts are frequently employed in sports analytics to compare the performance of athletes across various metrics such as speed, agility, and strength.
2. **Product comparison:** Businesses often use radar charts to compare the features or characteristics of various products, including those of competitors.
3. **Skills assessment:** Human resource departments can utilize radar charts to assess an individual's skills or competencies across multiple areas.
4. **Customer analytics:** Analysts might use a radar chart to visualize customer satisfaction ratings across dimensions, including product quality, customer service, and value for money.

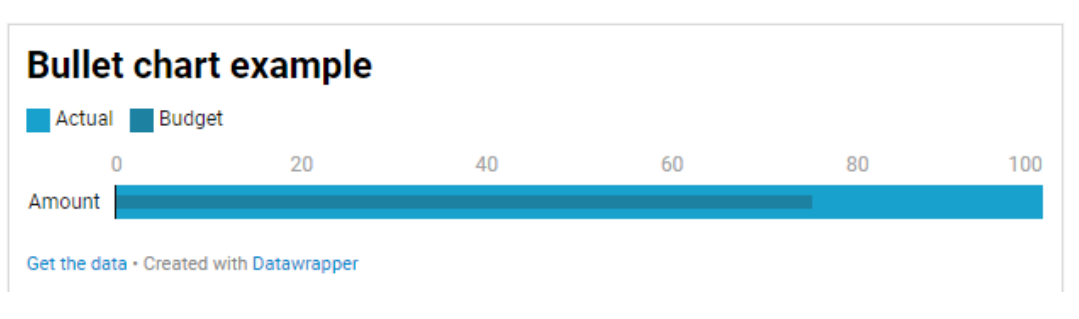
Above is an example of a chart I created using the **Chart.js** library.[6]

Chart.js is a library offering a comprehensive set of commonly used chart types, along with a range of plugins and customization options to cater to your needs. Beyond the standard set of charts, you can also leverage additional chart types maintained by the community. One of the notable features of Chart.js is its capability to combine different chart types into a combo chart, providing even more flexibility in how you present your data. Moreover, Chart.js supports custom plugins that enable you to incorporate various functionalities, such as annotations, zooming, and drag-and-drop, to enhance the interactivity and usability of your charts.

Here is the script code for the chart above. It was copied from the Chart.js site and adjusted with my sample data. It is also ready to be embedded on a website.

```
<html>
  <head>
    <title>Radar Chart Example</title>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/3.9.
  </head>
  <body>
    <canvas id="radarChart"></canvas>
    <script>
      var canvas = document.getElementById("radarChart");
      var chart = new Chart(canvas, {
        type: "radar",
        data: {
          labels: [
            'A',
            'B',
            'C',
            'D',
            'E',
            'F',
            'G'
          ],
          datasets: [{
            label: '2022',
            data: [65, 59, 90, 81, 56, 55, 40],
            fill: true,
            backgroundColor: 'rgba(255, 99, 132, 0.2)',
            borderColor: 'rgb(255, 99, 132)',
            pointBackgroundColor: 'rgb(255, 99, 132)',
            pointBorderColor: '#fff',
            pointHoverBackgroundColor: '#fff',
            pointHoverBorderColor: 'rgb(255, 99, 132)'
          }, {
            label: '2023',
            data: [48, 48, 80, 90, 66, 47, 30],
            fill: true,
            backgroundColor: 'rgba(54, 162, 235, 0.2)',
            borderColor: 'rgb(54, 162, 235)',
            pointBackgroundColor: 'rgb(54, 162, 235)',
            pointBorderColor: '#fff',
            pointHoverBackgroundColor: '#fff',
            pointHoverBorderColor: 'rgb(54, 162, 235)'
          }]
        }
      });
    </script>
  </body>
</html>
```

Bullet chart



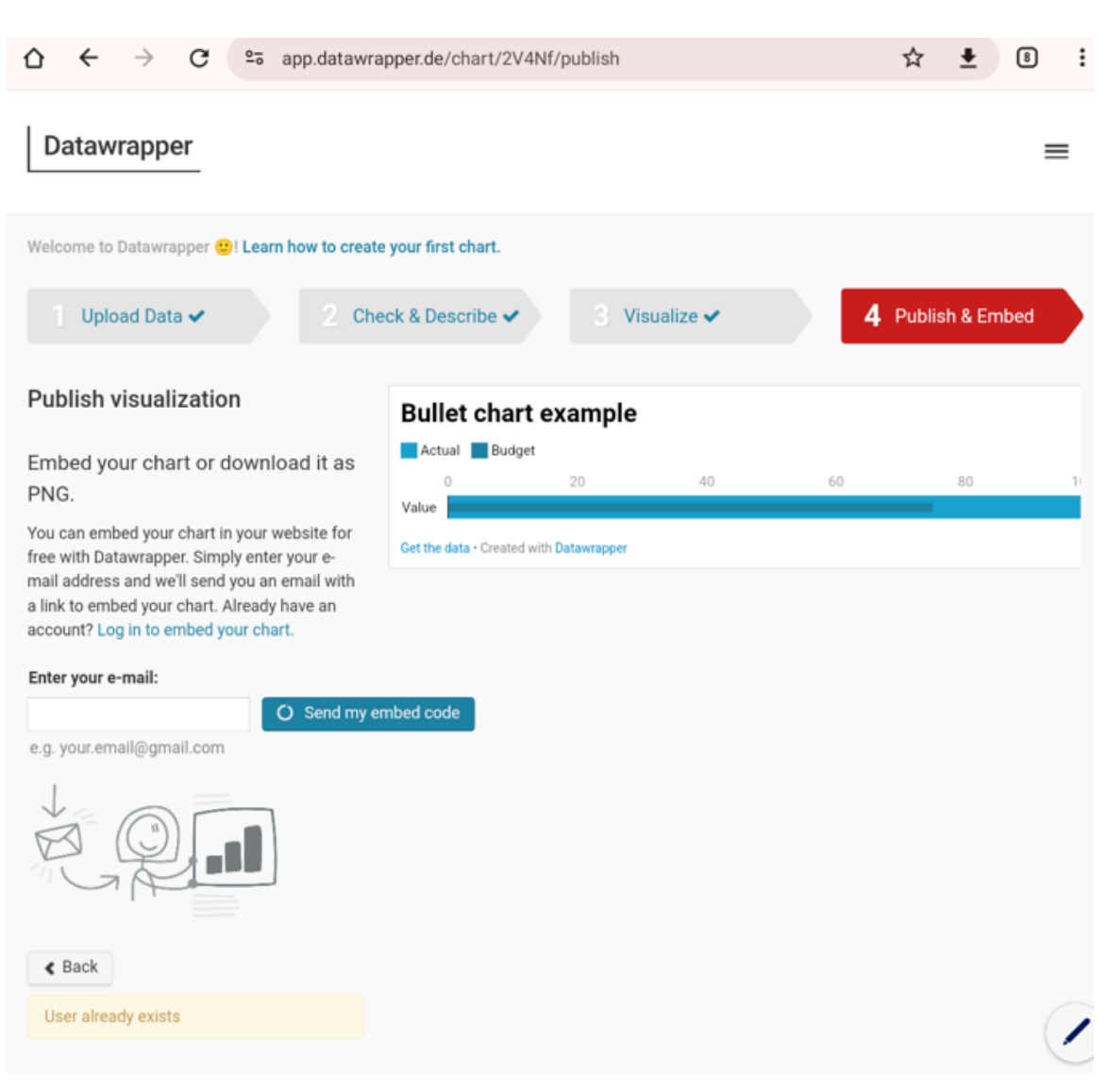
Bullet chart. Print screen by the author.

A **bullet chart** is a type of data visualization used to display performance data concerning predefined targets or benchmarks. It consists of the following elements:

1. A **bar** represents a single, primary measure (such as current performance or year-to-date revenue).
2. A **marker line** represents a comparative measure (such as a target or previous year's performance).
3. It can also show **ranges** that represent qualitative performance, such as poor, satisfactory, and sound, using different shades of a color or other colors.

The primary measure is presented against the comparative and the qualitative ranges to assess performance against goals or benchmarks quickly. **Bullet charts are often used in dashboards and reports for business performance management.**

Below is an example of a bullet chart I crafted using **Datawrapper**. Datawrapper is an online tool that enables users to create efficient, aesthetic and visually compelling visualizations. There is no need to have any specific IT knowledge such as learning HTML or Python – all can be done via a graphic interface. *Unfortunately*, it is a fully online tool. There is no locally installed option. So, in order to get a visual, you need to upload your data directly to the tool. Datawrapper allows to save visualizations as *.png files or embed them onto a website.



Welcome to Datawrapper 😊! Learn how to create your first chart.

Publish visualization

Embed your chart or download it as PNG.

You can embed your chart in your website for free with Datawrapper. Simply enter your e-mail address and we'll send you an email with a link to embed your chart. Already have an account? Log in to embed your chart.

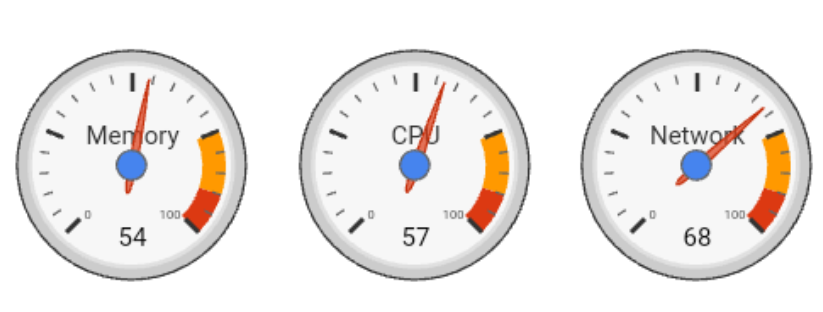
Enter your e-mail:

e.g. your.email@gmail.com

User already exists

Source: print screen by the author.

Gauges/ speedometers



Source: print screen by the author.

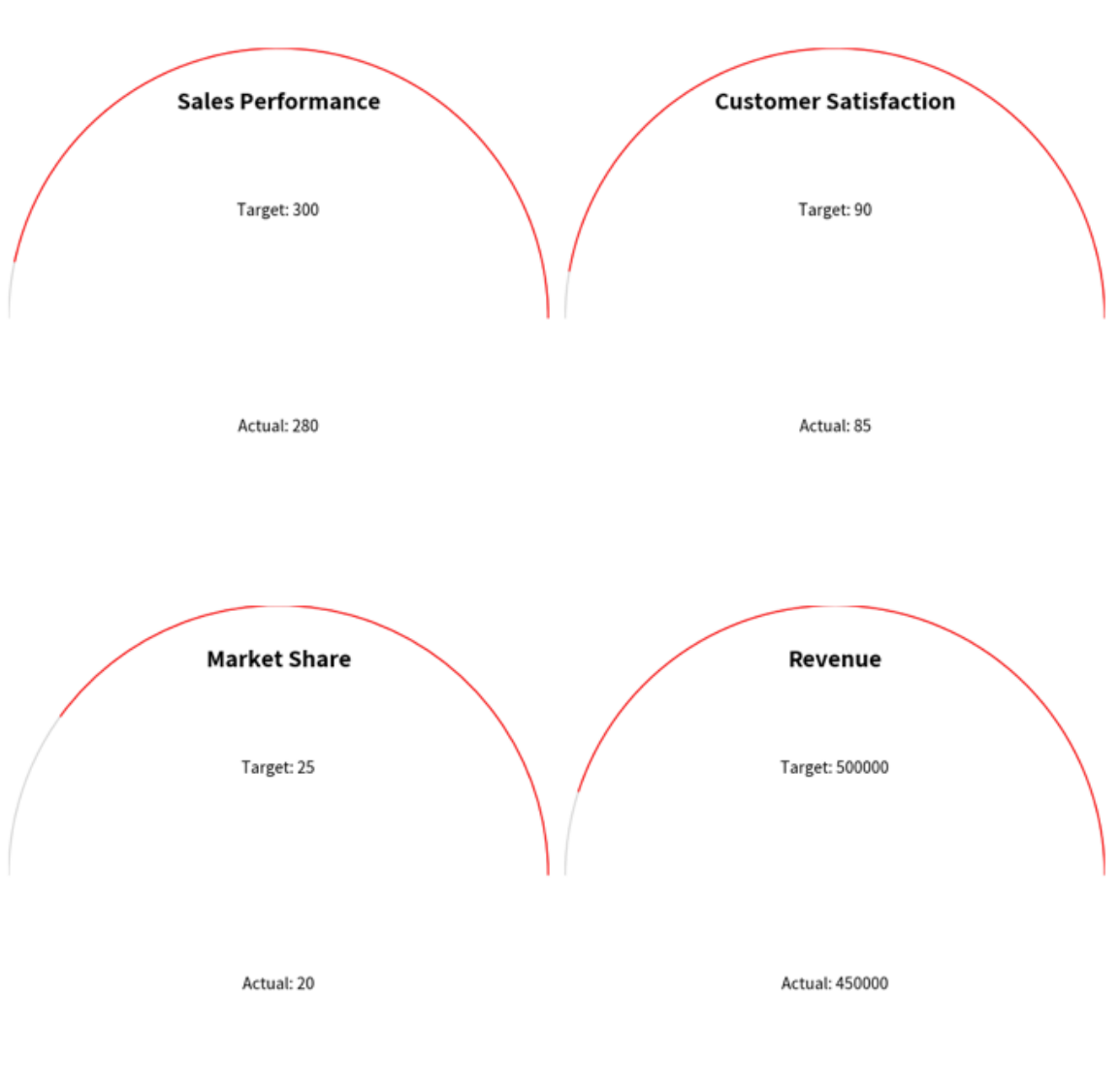
Gauges can serve a similar function as the bullet chart. A gauge or speedometer chart is a type of data visualization that resembles a car's speedometer and represents a value within a range of values. It's often employed to showcase key performance indicators (KPIs), performance against goals, or other data in a business context.

Here are the main components of a gauge chart:

1. **Needle or pointer:** Represents the data value in question.
    2. **Dial or scale:** A circular or semi-circular scale that displays the range of possible values.
    3. **Arcs or bands:** Different colored sections typically represent performance ranges.

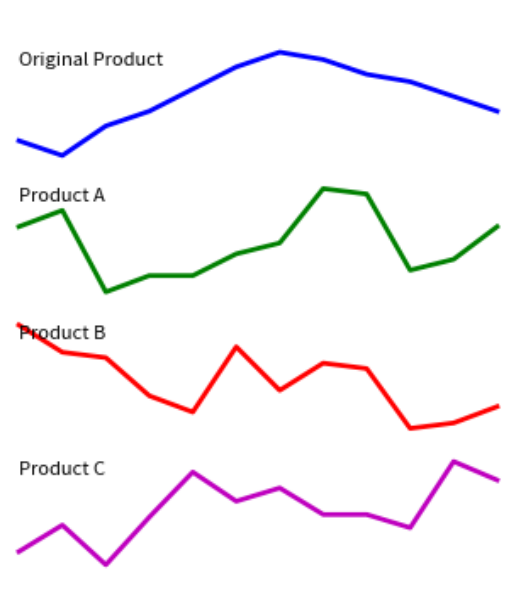
By looking at where the needle falls within the colored arcs or bands, viewers can quickly understand the value's context within the range and how it relates to predefined performance levels. This visual representation makes it easy to see if performance is on track, below expectations, or exceeding goals.[7]

You can easily prepare gauges like the ones given above using Google Charts. However, below is another example, this time created using ChatGPT and Noteable. Actually, this particular tool combination is not free, you can still generate code for such a visual directly in ChatGPT free version and then paste it yourself into a Noteable notebook.



Another type of gauges. Source: image generated by the author in ChatGPT & Noteable.

Mini charts or sparklines



Sparklines are compact charts designed to simultaneously display trends or variations across multiple data series, often within a spreadsheet cell or a small report section. Unlike standard charts, sparklines typically lack axis labels and legends. Their primary aim is to provide a clear and concise visual representation of data trends without consuming much space. Sparklines can effectively illustrate stock price trends, weather patterns, sales data, and more. I also use them as a navigational tool; users can click on a sparkline to access the full-sized chart if they spot something intriguing.[8]

Here are some scenarios where sparklines prove helpful:

1. **Sales reports:** Illustrate monthly sales trends for each product in a catalog or spreadsheet.
2. **Stock market analysis:** Display historical stock price trends for multiple companies in a financial report.
3. **Website analytics:** Show trends in website traffic or user engagement metrics.
4. **Inventory management:** Depict fluctuations in inventory levels across various products.

Above is an example of sparklines chart I created. For this specific exercise I used again ChatGPT with Noteable plugin on. This time, I have done that on a website opened on an Android device. I generated random sample data for the charts and the code for the chart themselves (code below).

```
import numpy as np

np.random.seed(0)
sales_a = [200 + np.random.randint(0, 21) for _ in range(12)]
sales_b = [300 - np.random.randint(0, 21) for _ in range(12)]
sales_c = [250 + np.random.randint(-20, 21) for _ in range(12)]

fig, axs = plt.subplots(4, 1, figsize=(4, 4))

axs[0].plot(df['Sales'], color='b', linewidth=2)
axs[0].axis('off')

axs[1].plot(sales_a, color='g', linewidth=2)
axs[1].axis('off')

axs[2].plot(sales_b, color='r', linewidth=2)
axs[2].axis('off')

axs[3].plot(sales_c, color='m', linewidth=2)
axs[3].axis('off')

plt.show()
```

Summary

This article aimed to expand the world of data visualizations beyond bar, line, and pie charts. There are many other, more advanced options worth considering for producing insightful and engaging visualizations. A byproduct to this was the review and "field-test" of various free tools, that one could use to produce such visualizations.

In particular, I discussed the use-cases, showed pros and cons and demonstrated examples of intricate charts such as Sankey diagrams, waterfall charts, radar charts, dumbbell plots, gauges, sparklines, and bullet charts. I used various tools to craft these visualizations, including Google Charts, Noteable, Python, and Plotly.

My primary goal was to help you to unlock the potential of these visualizations and enable you to weave a more compelling narrative with data. **Regardless of I succeeded or not, I encourage you to experiment with these alternative chart types to get an understanding of their characteristics, testing various applications, and further enhance your data-driven storytelling arsenal.**

> Did you like this article? Consider a subscription to get notified about new stories I publish, follow me or simply leave a 👏.

References

1. Wikipedia, Skid block
2. Michal Szudejko, Decalogue of visualization
3. Google Charts, Sankey diagram
4. Tessica Dall, Need to emphasize change in your dashboard? Forget bar charts, use dumbbell charts instead!
5. Elijah Meeks, Introducing Data Prism the automatic chart builder
6. Chart.js, Radar chart
7. Bernardita Calzon, Discover The Power Of Gauge Charts: Definition, Best Practices & Examples
8. Wikipedia, Sparkline

• • •

WRITTEN BY

Michal Szudejko

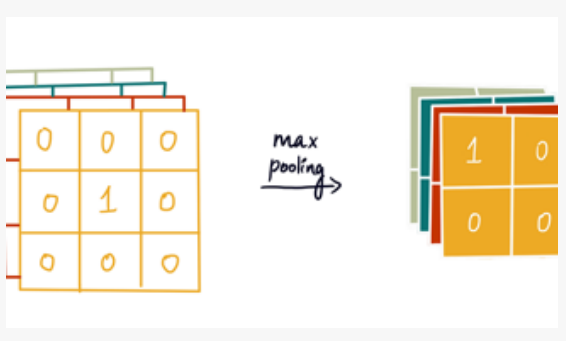
See all from Michal Szudejko

Topics:

Data Driven Storytelling
Data Science
Hands On Tutorials
Presentations
Visualization

Share this article: ○ ○ ○

Related Articles

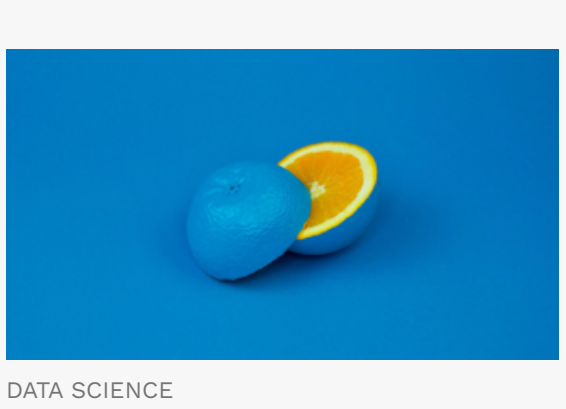


ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao
August 20, 2024 • 6 min read

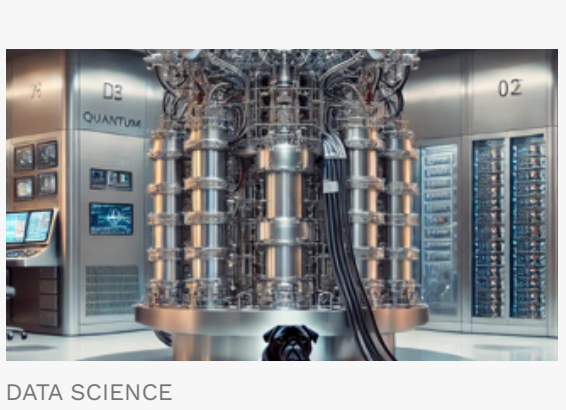


DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of…

Piero Paialunga
August 21, 2024 • 12 min read



DATA SCIENCE

Solving a Constrained Project Scheduling Problem with Quantum Annealing

Solving the resource constrained project scheduling problem (RCPSP) with D-Wave's hybrid constrained quadratic model (CQM)

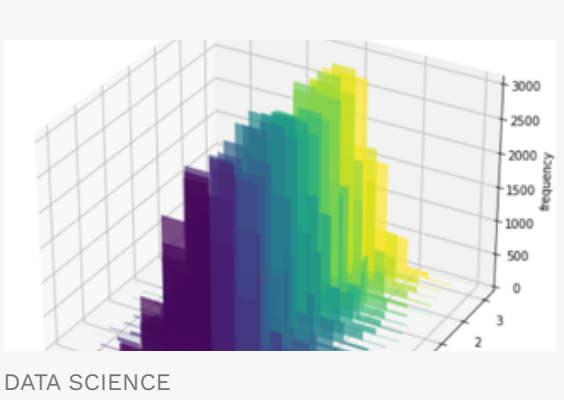
Luis Fernando PÉREZ ARMAS, Ph.D.
August 20, 2024 • 28 min read

DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao
February 3, 2023 • 6 min read



DATA SCIENCE

Must-Know in Statistics: The Bivariate Normal Projection Explained

Derivation and practical examples of this powerful concept

Luigi Battistoni
August 14, 2024 • 7 min read

DATA SCIENCE

How to Make the Most of Your Experience as a TDS Author

A quick guide to our resources and FAQ

TDS Editors
September 13, 2022 • 4 min read

DATA SCIENCE

Our Columns

Columns on TDS are carefully curated collections of posts on a particular idea or category…

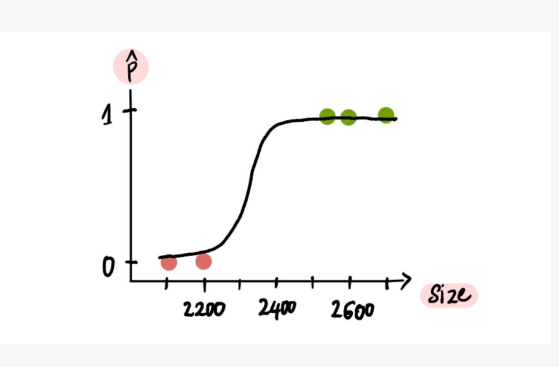
TDS Editors
November 14, 2020 • 4 min read

DATA SCIENCE

Optimizing Marketing Campaigns with Budgeted Multi-Armed Bandits

With demos, our new solution, and a video

Vadim Arzamasov
August 16, 2024 • 10 min read



DATA SCIENCE

Back to Basics, Part Tres: Logistic Regression

An illustrated guide to everything you need to know about Logistic Regression

Shreya Rao
March 2, 2023 • 8 min read

Job title*

Job level*

Please Select

Company name*

☐ I consent to receive newsletters and other communications from Towards Data Science publications.*

Subscribe Now